[18]

PyTorch for Classification

PyTorch for Classification

Welcome

Welcome! In this lesson, you'll learn how to build and train neural networks for classification tasks using the `PyTorch` library. Along the way, you'll use `PyTorch` to build, train, and test your own neural network on a real-world dataset of student grades, available under a CC-BY 4 license.

Unlike regression models, which try to predict numeric values (like what a student actually scored on an exam), classification models try to predict broader categories (for example, did a student pass or fail). In this lesson, we'll cover two types of classification:

- **binary classification**: models where there are only two possible outputs (pass or fail)
- **multiclass classification**: models where there are more than two possible outputs (poor, average, excellent)

Specifically, you'll learn how to

- prepare and encode categorical data
- use sigmoid activation and thresholds for binary classification
- use softmax and argmax for multiclass classification
- use cross-entropy to compute categorical loss
- evaluate network performance using accuracy and F1-scores

Prerequisites

This course assumes some prior experience using PyTorch to train neural networks for regression tasks. To get the most out of this course, you should also be familiar with the basics of ML (like train-test splits) and the ideas behind algorithms like gradient descent.

We will cover new algorithms conceptually, so there is no advanced mathematics needed to take the course. We will use sequential models, so there is also no need to know object-oriented programming.

Happy coding!

Instructions

We've loaded a basic classification model in the workspace for this exercise. The model tries to predict success in a course based on data about study habits!

Try playing around with different inputs to the model (or even use your own study habits – but know that this is a very simple model.)

When you are ready to learn how to build models like this yourself, select `Next`!

To motivate this course, we've trained a neural network to try to predict if a student will pass a course based on data about study habits!

First, run the cell below to import the libraries we'll use.

```
import pandas as pd
import torch
import numpy as np
```

We've created sample data for a fictitious student, and fed the sample data through our model to generate a prediction. Run the cell to see if the model predicts our sample student will pass.

Then, try changing the value of `study_alone` from `1` (studies alone) to `0` (studies in a group). Does group study improve the model's prediction?

Feel free to try out other sample inputs. What do you think our model cares about? Does the model's behavior make sense to you?

```
# student study habits

# how many hours does the student study per week?
# 0: None, 1: <5 hours, 2: 6-10 hours, 3: 11-20 hours, 4: > 20 hours
weekly_hours_studying =  2

# how much does the student take notes
```

```
# 0: Never take notes, 1: Sometimes take notes, 2: Always take notes
notetaking = 2

# does the student study alone (input 1) or in a group (input 0)
study_alone = 1

# Load information into a PyTorch tensor
sample_input = torch.tensor([weekly_hours_studying, notetaking, study_alone], dtype=torch.f

# Load our neural network model
welcome_model = torch.load('welcome_model.pth').eval()

# Generate prediction
def student_prediction(sample_input):
   with torch.no_grad():
       probability = np.round(welcome_model(sample_input).item(), 4)
       pct_pass = np.round(probability * 100, 2)
       prediction = f'The model predicts a ({pct_pass}%) probability of passing.'
   return prediction

student_prediction(sample_input)
```

'The model predicts a (61.8%) probability of passing.'

Note that our model is fairly simple, so its predictive power may be limited.
But don't worry! In the next exercises, you'll learn how to build much more powerful neural networks for both binary and multiclass classification tasks. Let's get started!

Encodings

Classification models output (or predict) categorical values often called **labels**. For example, in the table below we'd say that each student has been **labelled** as either `Passed` or `Failed`:

ID	High_School_Type	Letter_Grade	Outcome
1	State	A	Passed
2	Private	C	Passed
3	Other	F	Failed

The categories corresponding to these labels are called **classes** (which is where **classification** comes from!)
Importantly, these labels are categorical data, and not numeric data. But neural networks require both input features and labels to be numeric.
So before building and training our models, we'll need to convert the categorical data into a numeric format (a process called **encoding**).
There are two basic encoding techniques: **label encoding** and **one-hot encoding**.

Label Encoding

Label encoding maps categories to numbers directly (typically to integers).
For example, the `Outcome` column has two categorical values: `Passed` and `Failed`. We can label encode `Outcome` by replacing `Passed` with `1` and `Failed` with `0`.

Here's the pandas code:

```
df['Outcome'] = df['Outcome'].replace(
    {'Passed':1,
    'Failed':0})
```

Copy to Clipboard

Label encoding is often used for categories that are already *meaningfully ordered* in some way.

For example, the `Letter_Grade` column is ordered from the highest grade A to the lowest grade F. We can numerically encode these grades from 4 to 0:

```
df['Letter_Grade'] = df['Letter_Grade'].replace(
    {'A':4,
    'B':3,
    'C':2,
    'D':1,
    'F':0})
```

Copy to Clipboard

Here's the outcome of these two label encoding processes:

Original Columns | **Label Encoded Columns**

ID	Letter_Grade	Outcome	ID	Letter_Grade	Outcome
1	A	Passed	1	4	1
2	C	Passed	2	2	1
3	F	Failed	3	0	0

One-Hot Encoding

What about the `High_School_Type` column? This column has three categorical values: `State`, `Private`, and `Other`. There's no innate ordering to these categories, making label encoding less effective.

In this case, we'll use a technique called **one-hot encoding**.

Instead of replacing categories with numbers, we'll create a new binary column for each category.

Let's go through an example to understand how this works.

Since `High_School_Type` has three categorical values, we'll create three new *columns*, one for each category:

Original Column | **One-Hot Encoded Columns**

ID	High_School_Type	ID	State	Private	Other
1	State	1	-	-	-
2	Private	2	-	-	-
3	Other	3	-	-	-

The first student in our dataset attended `State` high school. So in the first row, corresponding to that student, we'll put a `1` in the `State` column and `0` in the other columns.

Original Column		One-Hot Encoded Columns			
ID	High_School_Type	ID	State	Private	Other
1	State	1	1	0	0
2	Private	2	-	-	-
3	Other	3	-	-	-

The second student attended `Private` highschool. So in that second row, we'll put a `1` in the `Private` column and a `0` in the others.

Original Column		One-Hot Encoded Columns			
ID	High_School_Type	ID	State	Private	Other
1	State	1	1	0	0
2	Private	2	0	1	0

3	Other	3	-	-	-

The third student attended `Other`, so we'll put a `1` in `Other`:

Original Column		One-Hot Encoded Columns			
ID	**High_School_Type**	**ID**	**State**	**Private**	**Other**
1	State	1	1	0	0
2	Private	2	0	1	0
3	Other	3	0	0	1

Pandas provides a `pd.get_dummies()` method to automate one-hot encoding. The syntax is

```
df = pd.get_dummies(
    df,
    columns=['High_School_Type'],
    dtype=int)
```

Copy to Clipboard

where

the `columns` parameter is a list of the columns we want to one-hot encode

`dtype=int` tells pandas to use `1` and `0` for the encoding

Instructions

Checkpoint 1 Passed

1.

We've loaded a Jupyter Notebook in the exercise workspace. Within the Jupyter Notebook, there are text instructions as well as code cells for writing and executing Python code.

Some of those code cells are labeled **Checkpoint**, and correspond to these tasks! When you reach a checkpoint, follow the instructions to write code in the next code cell. When you think you've got it, run the checkpoint code cell, save the notebook, and press `Test Work` to be evaluated by our code testing system.

If your code is correct, the box at the top of these instructions will get a checkmark. If not, an error message and hint will display over the notebook.

Ready? Let's go through an example step-by-step.

Locate **Checkpoint 1/3** in the notebook.

Select the next code cell and type `import pandas as pd`.

Click the `Run` button or the `Shift`+`Enter/Return` keys (see image below)

Click the `Save` button or use the `Control/Command`+`S` keys to save your work

Click the `Test Work` button below the Jupyter Notebook to check if you've completed the Checkpoint!

Checkpoint 2 Passed

2.

First, run through the example cells below Checkpoint 1 and feel free to play around with different encodings!

The categorical columns

`Additional_Work`

`Regular_Artistic_or_Sports`, and

`Has_Partner`

contain binary **Yes** or **No** values in the DataFrame `df`.

Label-encode all three columns. That is, convert all **Yes** values to **1** and all **No** values to **0** in each column.

Perform the label-encoding in place. That is, after label encoding, the original columns should contain the label encoded data (as opposed to creating new columns to store the label encoded data).

	Student_ID	Additional_Work	Regular_Artistic_or_Sports	Has_Partner
1		Yes	No	No
2		Yes	Yes	Yes
3		No	Yes	No
4		Yes	No	Yes
5		No	No	Yes

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Use pandas `.replace()` method, passing in a dictionary. The keys of the dictionary are the original values, the values of the dictionary are the encoded values:

```
.replace({'original':encoded})
```

Copy to Clipboard

When we say in-place, we mean to call replace as follows:

```
df['col'] = df['col'].replace({'original':encoded})
```

Copy to Clipboard

This way, the encoded values are in the original column, instead of existing in a new column.

Checkpoint 3 Passed

3.

The column `MT1_Preparation` contains categorical data on how students prepared for the first midterm of the course. Students may have either prepared "Alone", "With Friends", or "Not Applicable" if they didn't prepare at all.

	Student_ID	High_School_Type	MT1_Preparation
1		State	Alone

2	Private	Alone
3	Other	With Friends
4	State	Alone
5	State	Not Applicable

One-hot encode the `MT1_Preparation` column in the DataFrame `df`. Be sure to return the one-hot encodings as integers and save the one-hot encoded DataFrame back to the variable `df`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
Use the `pd.get_dummies()` method with inputs:
the original DataFrame
`columns=['column_to_encode']`
`dtype=int`

Checkpoint 1/3
Follow the instructions beneath the exercise narrative to import pandas and practice testing your work with our code testing system!

```
import pandas as pd
```

Example - Label Encoding
Run the cell below to label-encode the `Letter_Grade` and `Outcome` columns of the tabel below:

Student_ID	Letter_Grade	Outcome
1	A	Passed
2	C	Passed
3	F	Failed
4	B	Passed
5	D	Failed

```
# create the sample dataframe
df = pd.DataFrame({'Student_ID':[1,2,3,4,5],
                   'Letter_Grade':['A','C','F','B','D'],
                   'Outcome':['Passed','Passed','Failed','Passed','Failed']})

# Label encode Letter_Grade and Outcome columns
df['Letter_Grade'] = df['Letter_Grade'].replace(
   {'A':4,
   'B':3,
   'C':2,
   'D':1,
   'F':0})

df['Outcome'] = df['Outcome'].replace(
   {'Passed':1,
    'Failed':0})

df.head()
```

	Student_ID	Letter_Grade	Outcome
0	1	4	1
1	2	2	1
2	3	0	0
3	4	3	1
4	5	1	0

Example - One-Hot Encoding

Run the cell below to one-hot encode the `High_School_Type` column of the table below.

Student_ID	High_School_Type
1	State
2	Private
3	Other
4	State
5	State

```
# create the sample dataframe
df = pd.DataFrame({'Student_ID':[1,2,3,4,5],
                   'High_School_Type':['State','Private','Other','State', 'State']})

# One-hot encode High_School_Type column
df = pd.get_dummies(
   df,
   columns=['High_School_Type'],
   dtype=int)

df.head()
```

Checkpoint 2/3

The categorical columns

- `Additional_Work`
- `Regular_Artistic_or_Sports`, and
- `Has_Partner`

contain binary **Yes** or **No** values in the DataFrame `df`.

Label-encode all three columns. That is, convert all **Yes** values to **1** and all **No** values to **0** in each column.

Perform the label-encoding in place. That is, after label encoding, the original columns should contain the label encoded data (as opposed to c reating new columns to store the label encoded data).

Student_ID	Additional_Work	Regular_Artistic_or_Sports	Has_Partner
1	Yes	No	No
2	Yes	Yes	Yes

3	No	Yes	No
4	Yes	No	Yes
5	No	No	Yes

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# create the dataframe df
df = pd.DataFrame({'Student_ID':[1,2,3,4,5],
                   'Additional_Work':['Yes','Yes','No','Yes','No'],
                   'Regular_Artistic_or_Sports':['No','Yes','Yes','No','No'],
                   'Has_Partner':['No','Yes','No','Yes','Yes']})

## YOUR SOLUTION HERE ##
df = pd.get_dummies(
   df,
   columns = ['Additional_Work', 'Regular_Artistic_or_Sports', 'Has_Partner'],
   dtype = int, drop_first=True)

# show encoded output
df.head()
```

	Student_ID	Additional_Work_Yes	Regular_Artistic_or_Sports_Yes	Has_Partner_Yes
0	1	1	0	0
1	2	1	1	1
2	3	0	1	0
3	4	1	0	1
4	5	0	0	1

Checkpoint 3/3

The column `MT1_Preparation` contains categorical data on how students prepared for the first midterm of the course. Students may have either prepared "Alone", "With Friends", or "Not Applicable" if they didn't prepare at all.

Student_ID	High_School_Type	MT1_Preparation
1	State	Alone
2	Private	Alone
3	Other	With Friends
4	State	Alone
5	State	Not Applicable

One-hot encode the `MT1_Preparation` column in the DataFrame `df`. Be sure to return the one-hot encodings as integers and save the one-hot encoded DataFrame back to the variable `df`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# create the dataframe df
df = pd.DataFrame({'Student_ID':[1,2,3,4,5],
                   'MT1_Preparation':['Alone', 'Alone', 'With Friends', 'Alone', 'Not Applicable']})
```

```
## YOUR SOLUTION HERE ##
df = pd.get_dummies(
    df,
    columns = ['MT1_Preparation'],
    drop_first=True,
    dtype=int
)

# show encoded output
df.head()
```

	Student_ID	MT1_Preparation_Not Applicable	MT1_Preparation_With Friends
0	1	0	0
1	2	0	0
2	3	0	1
3	4	0	0
4	5	1	0

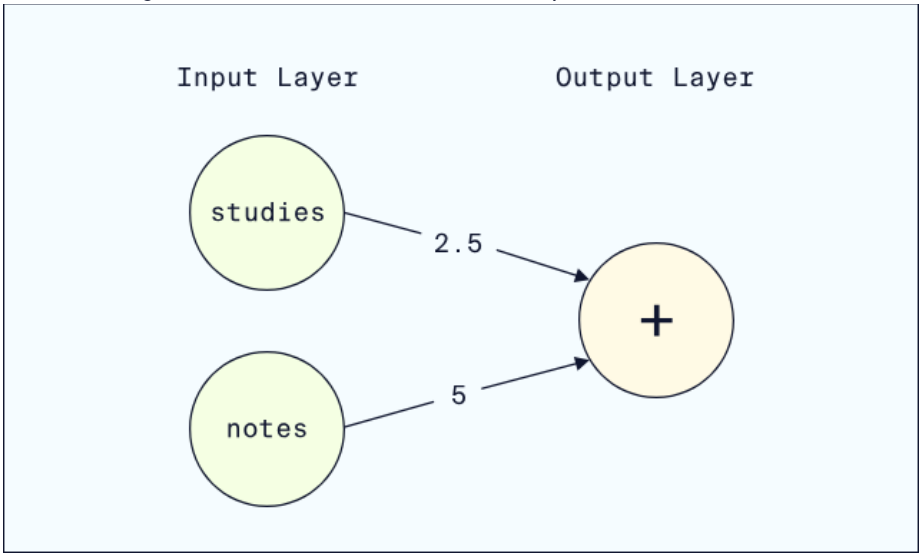
Moving Forward
For the rest of the lesson, we'll work with the full student performance dataset, with all these encodings (and more) already applied.

Sigmoids and Thresholds

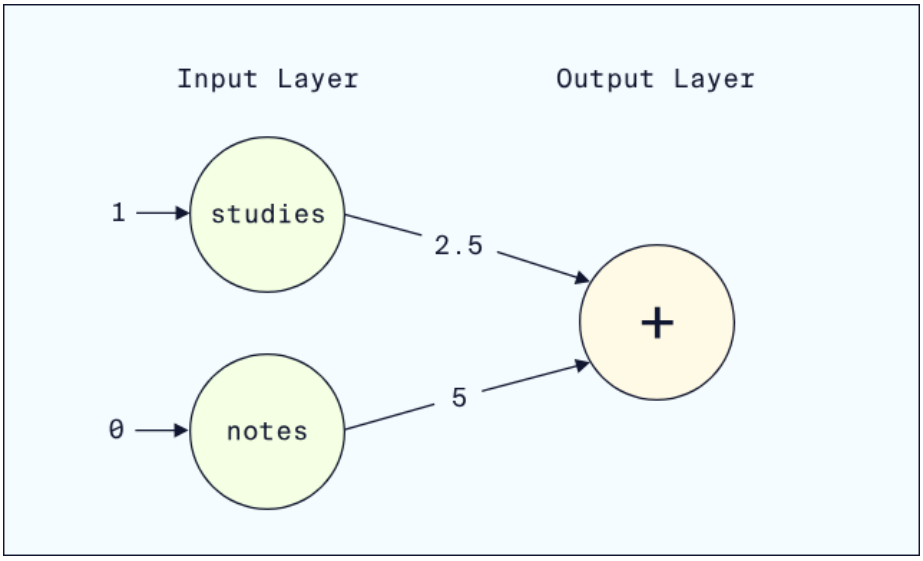
Suppose we want to predict whether a student will pass (output `1`) or fail (output `0`) an exam.
For this example, let's assume we have two input features:

- whether the student *studies* (`1` if they do, `0` if they do not)
- whether the student takes *notes* (`1` if they do, `0` if they do not)

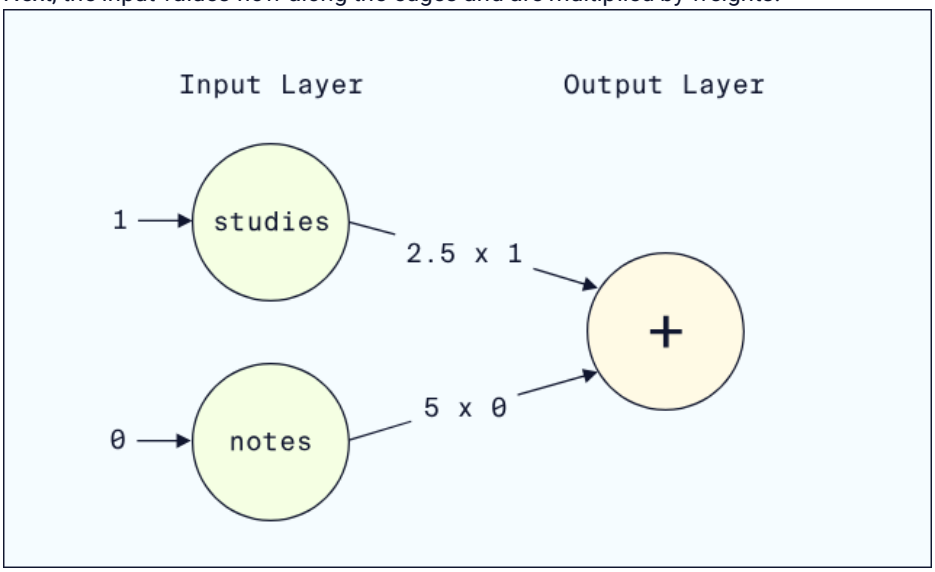
Let's add weights for each feature, and create a simple neural network model:



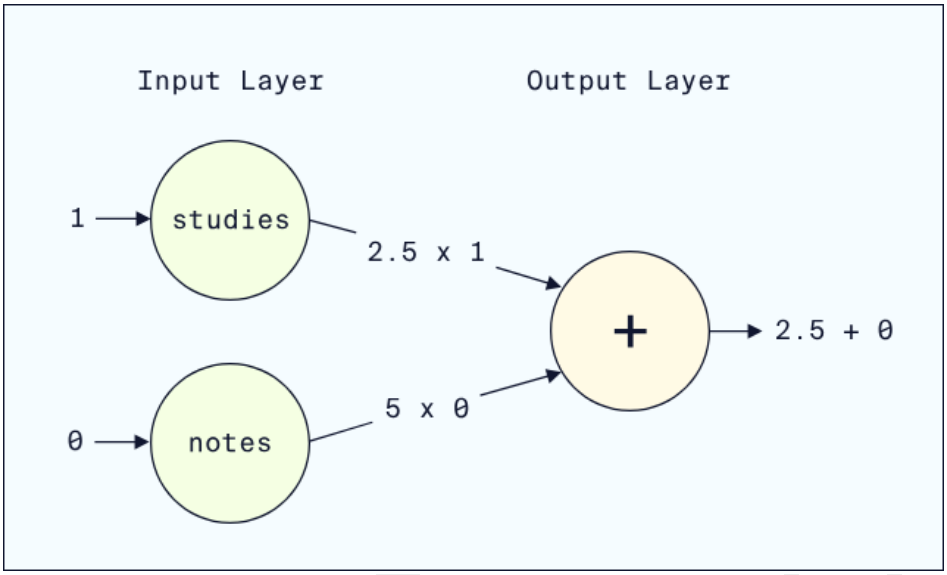
Suppose we want a prediction for a student who studies but does not take notes.
First, we input this student's data:



Next, the input values flow along the edges and are multiplied by weights:



Lastly, the output node adds up the weighted inputs:



The linear output for our student is `2.5`. But our goal is to predict a pass (`1`) or fail (`0`), so we aren't quite done yet.

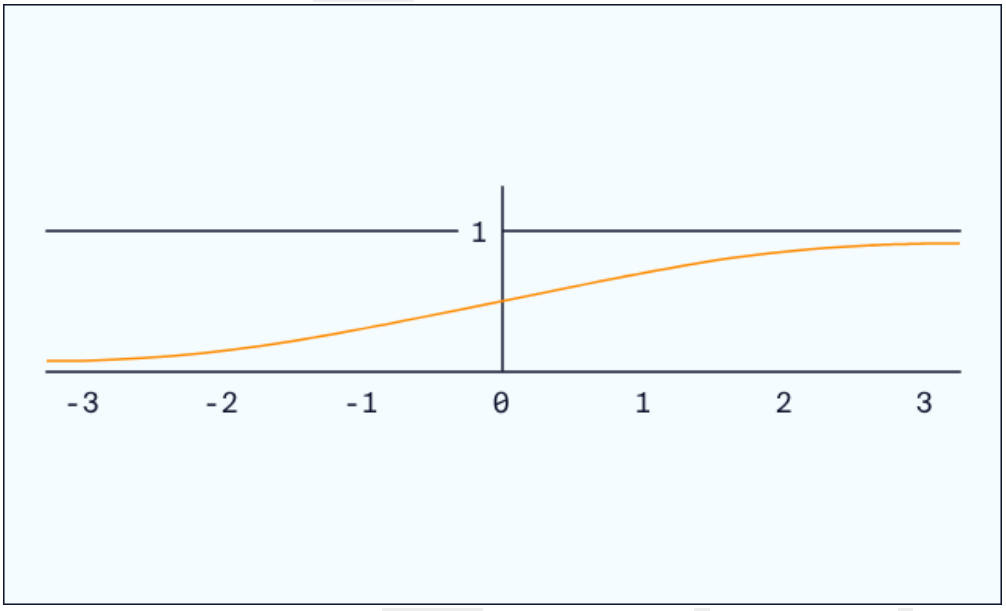
Sigmoid Activation Function

We need to convert `2.5` into either `1` or `0`. We'll start by using the **sigmoid activation function** to convert `2.5` into a **probability**, a value *between* `0` and `1`.

Mathematically, sigmoid is defined by the formula

$$sigmoid(x)=1 / (1+e^{-x})$$

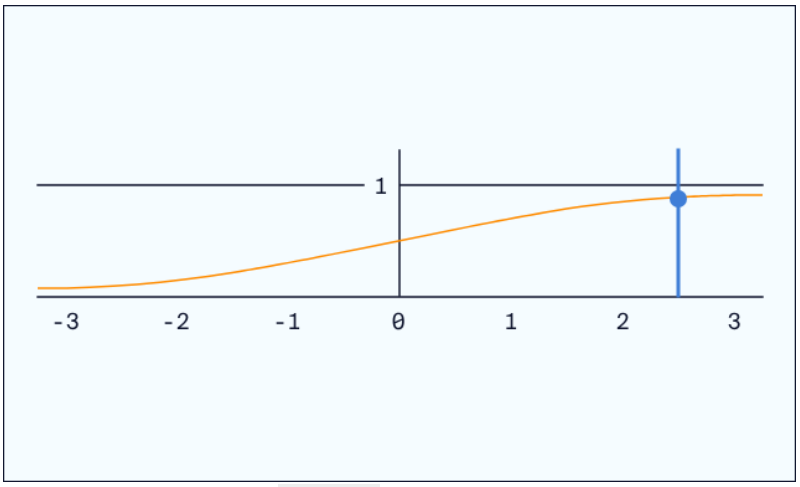
But the key to understanding `sigmoid` is the shape of its graph:



As inputs increase along the x-axis, `sigmoid` goes from close to `0` to being close to `1`. In other words, `sigmoid`

- takes any real number as input
- outputs a value between 0 and 1 (a probability!)

Our output was `2.5`. If we find `2.5` on the x-axis, the height of the curve (very close to `1`) is the output of `sigmoid`:

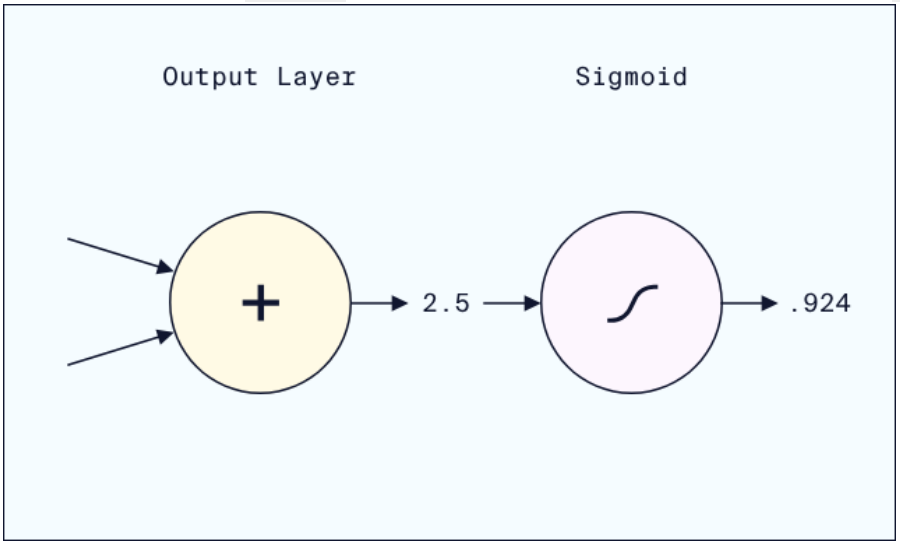


The exact output value of `sigmoid` is

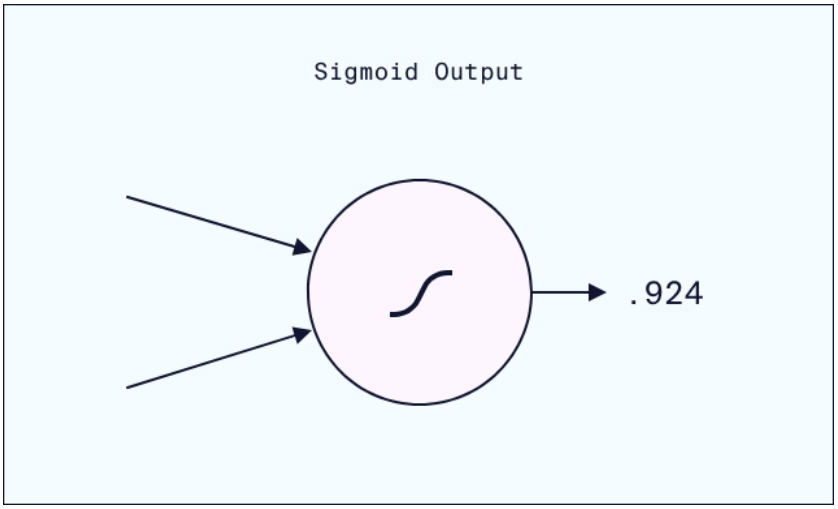
$$\text{sigmoid}(2.5) = \frac{1}{1+e^{-2.5}} \approx .924$$

$$\text{sigmoid}(2.5) = \frac{1}{1+e^{-2.5}} \approx .924$$

In other words, adding `sigmoid` activation to the output node gives us an output of `.924`:



Typically, we don't think of `sigmoid` as a separate node, but rather as an activation function applied to the output node:



Lastly, we can interpret this probability as saying that there is a 92.4% chance the student will pass the exam.

Threshold

We still need to convert the probability (`.924`) into a binary prediction (`1` or `0`). Since `.924` is so close to `1`, it would make sense to predict `1`. But what about an output of `.51`? We need to define a consistent **threshold** for our model:

if the output probability is at or above the threshold, predict `1`
otherwise, predict `0`

A common choice is to set the threshold at `.5`. However, for some models that may change. For example, a model for diagnosing a medical condition might set the threshold lower, to err on the side of caution.

In our example, a threshold of `.5` makes sense, resulting in a final prediction of `1`: our model expects this student to pass!

A quick way to perform this threshold computation in Python is

```
classification = int(probability >= threshold)
```

Copy to Clipboard

where

`probability >= threshold` returns `True` if the probability is above the threshold
`int()` converts the boolean `True/False` output to `1/0`

Instructions

Checkpoint 1 Passed

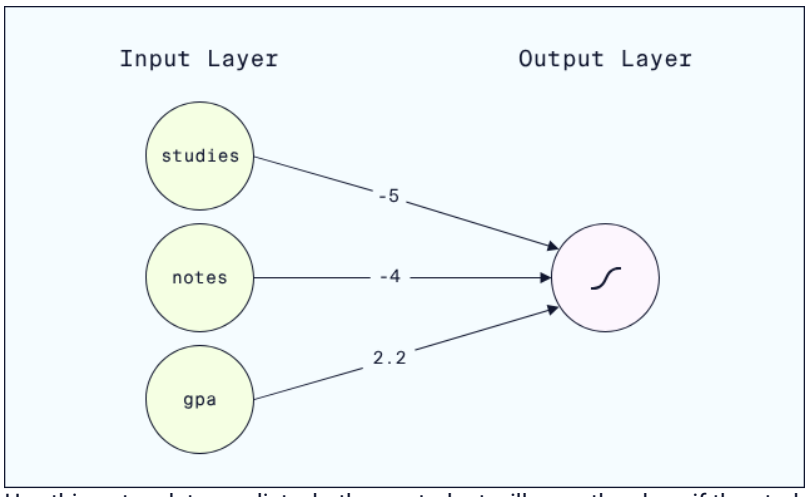
1.

First, run the `Setup` cell to define the `sigmoid` function. Then, run through the `Example` cell, which implements our example prediction above in Python.

For this Checkpoint, let's add the input feature `gpa` to our network, containing the student's GPA from the prior semester.

Let's also randomly initialize some weights (we'll train networks later in the course to optimize the weights):

`studies` weight is now `-5.0`
`notes` weight is now `-4.0`
`gpa` weight is `2.2`



Use this network to predict whether a student will pass the class if the student

- studies
- does not take notes
- has a `3.0` GPA from the prior semester

Set the classification threshold to `0.50`.
Save the predicted classification to the variable `classification`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
Start with the code from the `Example` cell and modify it for the new network.
To add in `gpa`, you'll need a `gpa` input and a `weighted_gpa` to add to the weighted sum.
Checkpoint 2 Passed

2.

In the last checkpoint, we used a threshold of `.5` and produced a classification of `1` (in other words, we predicted the student would pass.)
Re-do that calculation, but with a threshold of `.85`. Does the new threshold alter the classification? Why?
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
Take the threshold code from the prior checkpoint, and modify the `threshold` variable.
Checkpoint 3 Passed

3.

PyTorch implements sigmoid in `torch.nn.Sigmoid()`.
In the cell below, we've used `torch.nn.Sequential()` to implement the neural network from this exercise's example.

Need a Sequential refresher?

PyTorch's Sequential method lets us quickly build neural networks where the data flows sequentially from the inputs through to the outputs. The inputs to `Sequential` are linear layers and activation functions. For example, the code below defines a neural network with two layers of nodes:

- an input layer with 3 nodes
- a hidden layer with 2 nodes and ReLU activation
- an output layer with 1 node

```
model = nn.Sequential(
    nn.Linear(3,2) # data flows from input layer to 2-node hidden layer
    nn.ReLU() # ReLU activation is applied to the 2-node hidden layer
    nn.Linear(2,1) # data flows from the 2-node hidden layer to the output node
    )
```

Copy to Clipboard

Add `Sigmoid()` as a final activation function in the sequential network.

Add `nn.Sigmoid()` after the output layer is reached in `nn.Sequential()`. Remember to put a comma between inputs to `nn.Sequential`!

Setup

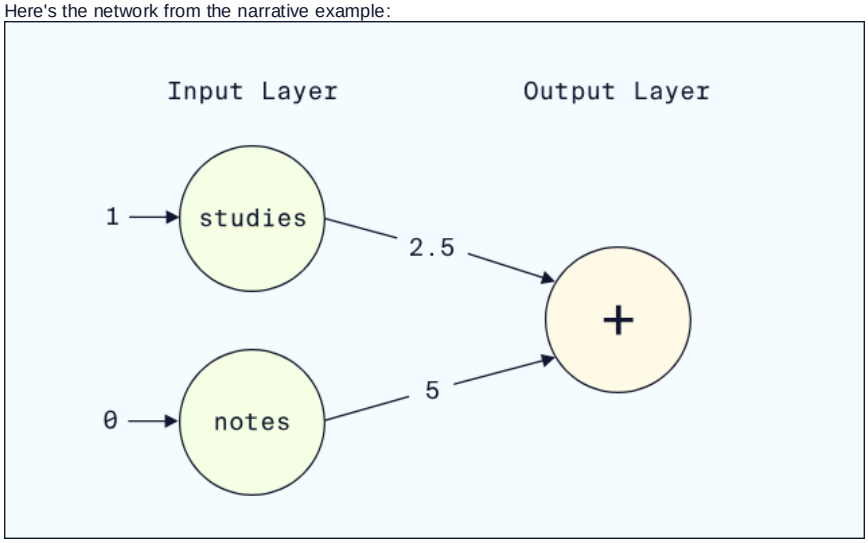
Run the next code cell to define `sigmoid`.

```
# import Python's math module to access the exponential function math.exp
import math

# define sigmoid
def sigmoid(x):
   return 1 / (1 + math.exp(-x))
```

Example: Binary Classification

Here's the network from the narrative example:



In the next code cell, we've implemented the classification process described in the exercise narrative. Run the cell to review the prediction! Feel free to alter the inputs to find out what this neural network would predict for different students.

```
# Define inputs for a student who studies but does not take notes
studies =  1.0
notes = 0.0

# Multiply each input by the corresponding weight
weighted_studies = 2.5 * studies
weighted_notes = 5.0 * notes

# Calculate weighted sum - this should produce 2.5 as in the narrative
weighted_sum = weighted_studies + weighted_notes
print("Weighted Sum:", weighted_sum)

# Apply the sigmoid activation function (run the setup cell if you haven't yet)
predicted_probability = sigmoid(weighted_sum)

# Determine a prediction using a threshold of .5
threshold = 0.50
classification = int(predicted_probability >= threshold)

# Print probability and classification
print("Probability:", predicted_probability)
print("Classification:", classification)
```

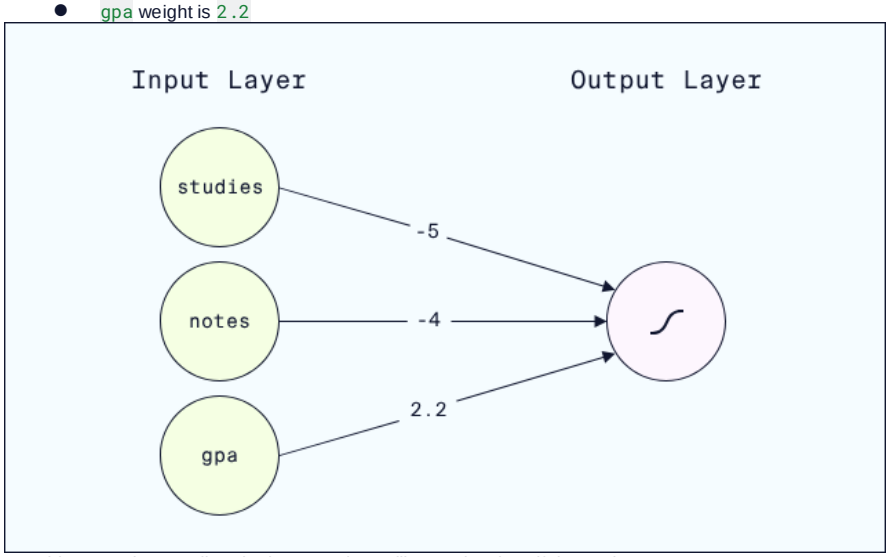
Weighted Sum: 2.5
Probability: 0.9241418199787566
Classification: 1

Checkpoint 1/3

Let's add the input feature `gpa` to our network, containing the student's GPA from the prior semester.
Let's also randomly initialize some weights (we'll train networks later in the course to optimize the weights):

- `studies` weight is now `-5.0`

- `notes` weight is now `-4.0`
- `gpa` weight is `2.2`



Use this network to predict whether a student will pass the class if the student

- studies
- does not take notes
- has a `3.0` GPA from the prior semester

Set the classification threshold to `0.50`.
Save the predicted classification to the variable `classification`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##

# Define inputs
studies = 1.0
notes = 0.0
gpa = 3.0

# Multiply each input by the corresponding weight
weighted_studies = -5.0 * studies
weighted_notes = -4.0 * notes
weighted_last_gpa = 2.2 * gpa

# Calculate weighted sum
weighted_sum = weighted_studies + weighted_notes + weighted_last_gpa

# Apply the sigmoid activation function (run the setup cell if you haven't yet)
predicted_probability = sigmoid(weighted_sum)

# Determine a prediction using a threshold of .5
threshold = 0.50
classification = int(predicted_probability >= threshold)

# Print probability and classification
print("Probability:", predicted probability)
print("Classification:", classification)
```

Probability: 0.8320183851339246
Classification: 1

Checkpoint 2/3
In the last checkpoint, we used a threshold of `.5` and produced a classification of `1` (in other words, we predicted the student would pass.)
Re-do that calculation, but with a threshold of `.85`. Make sure to save your final classification to the variable `classification`.
Does the new threshold alter the classification? Why?
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# re-calculating the probability from the prior checkpoint for ease
predicted_probability = sigmoid(1.6)

## YOUR SOLUTION HERE ##
threshold = 0.85
classification = int(predicted_probability >= threshold)
```

```
# Print probability and classification - do not modify
print("Probability:", predicted probability)
print("Classification:", classification)
```
Probability: 0.8320183851339245
Classification: 0

Checkpoint 3/3
PyTorch implements sigmoid in `torch.nn.Sigmoid()`.
In the cell below, we've used `torch.nn.Sequential()` to implement the neural network from this exercise's example.
Need a Sequential refresher?
PyTorch's Sequential method lets us quickly build neural networks where the data flows sequentially from the inputs through to the outputs. The inputs to `Sequential` are linear layers and activation functions. For example, the code below defines a neural network with two layers of nodes:

- an input layer with 3 nodes
- a hidden layer with 2 nodes and ReLU activation
- an output layer with 1 node

```
model = nn.Sequential(
   nn.Linear(3,2) # data flows from input layer to 2-node hidden layer
   nn.ReLU() # ReLU activation is applied to the 2-node hidden layer
   nn.Linear(2,1) # data flows from the 2-node hidden layer to the output node
         )
```
Add `Sigmoid()` as a final activation function in the sequential network.
```
from torch import nn

## YOUR SOLUTION HERE ##
model = nn.Sequential(
   nn.Linear(2,1),
   nn.Sigmoid())
```

Binary Cross-Entropy Loss

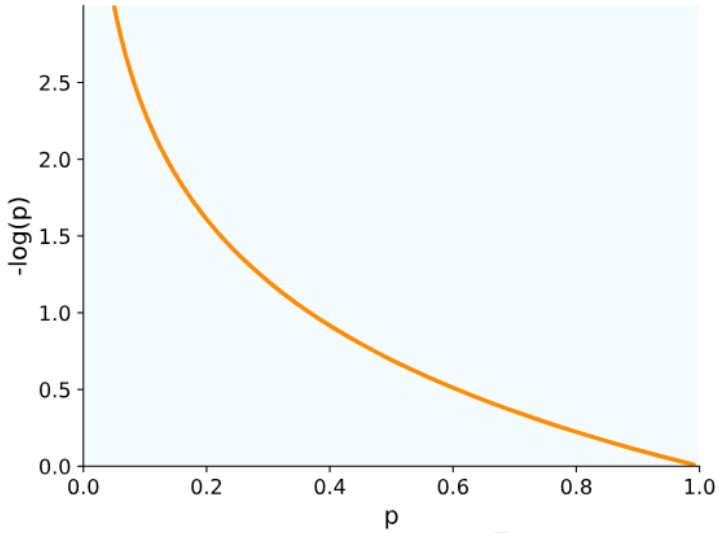
Our binary classification model takes student data as input and classifies each student as either passing/`1` or failing/`0`.
To train the model, we need a **loss function** to measure how well our predicted classifications match the true classifications from our training dataset. The loss function will output a high value if our predictions are bad, and a low value if our predictions are good.
One of the most common
loss functions
Preview: Docs Loading link description
for binary classification is **binary cross-entropy loss (BCELoss)**.
To compute BCELoss, we need to analyze two different cases:
the true classification is `1`
the true classification is `0`
Note that the true classification is also sometimes called the true class or the true label, the target class or the target label. (We also wish machine learning engineers had settled on just one term!)

Case 1: true classification `1`

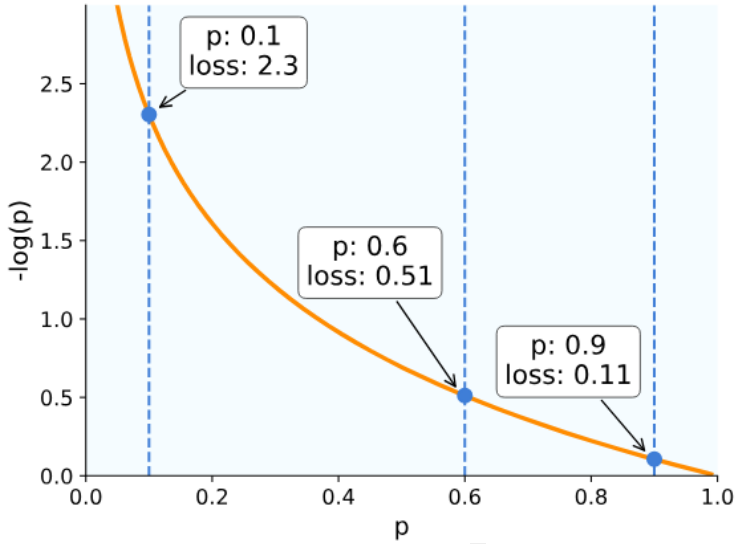
Before applying a threshold, our model outputs a sigmoid probability between `0` and `1`.
Let's call this output probability `p`.
If the true classification is `1`, the BCE Loss is computed using the negative logarithm:

$$\text{BCELoss}(p) = -\log(p)$$

Why the negative logarithm? There are a few theoretical reasons that we won't cover in this course. But another reason is more straightforward. The negative logarithm just has the right shape:



Remember, we are assuming the true classification is `1`. Probabilities close to `1` produce a very low output from the negative logarithm. Probabilities close to `0` (the wrong prediction) produce a larger output from the negative logarithm.
For example, here is the loss computation for probabilities of `p = .9`, `p = .6`, and `p=.1`:



As our predicted probability gets further away from `1`, and thus more and more wrong, the loss increases (indicating larger and larger error).

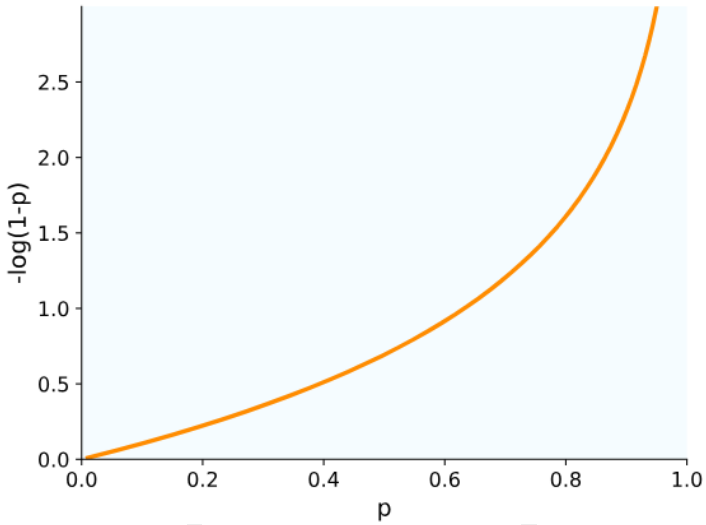
Case 2: true classification `0`

In this case, we will still use the negative logarithm but with a slight adjustment. Instead of applying the negative logarithm to `p`, we'll apply it to `1-p`:

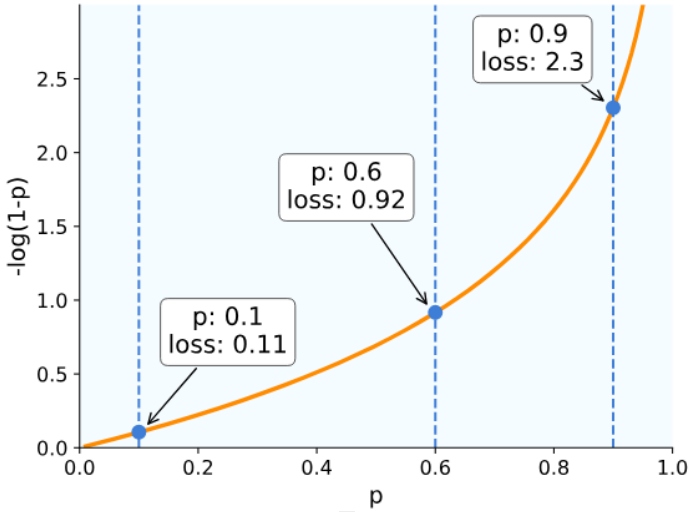
$$\text{BCELoss(p)} = -\log(1-\text{p})$$

$$\text{BCELoss}(p) = -\log(1-p)$$

Inserting `1-p` flips the shape of the negative logarithm curve:



Probabilities close to `0` now result in very low loss, since `0` is now the true classification.
On the other hand, probabilities close to `1` now produce a high loss, since `1` is now the incorrect classification.
Let's compute this version of BCELoss for the same three example points:



Now, as our probability gets closer to `0` it gets more and more correct, resulting in lower and lower loss values!

Combining the two cases

To compute loss for multiple data points with different true classifications, we would:

- split the data points into those with true classification `1` and those with true classification `0`
- for datapoints with true classification `1`, compute the loss as `-log(p)`
- for datapoints with true classification `0`, compute the loss as `-log(1-p)`
- collect all the loss values and compute the average loss

Instructions

Checkpoint 1 Passed

1.

First, run the setup cell and experiment with different values of `p` and the true classification `y`.
Now, suppose a network outputs a probability `p = .85`. If the BCELoss for this prediction is fairly low, what is the true classification?
Uncomment the line in the next cell corresponding to the answer to this question.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
A prediction of `.85` is a fairly confident prediction of `1` as the true classification. If this confident prediction is correct, should the loss be high or low?

Checkpoint 2 Passed

2.

PyTorch implements BCELoss in `torch.nn.BCELoss()`. Here's an example:

```
import torch
from torch import nn

# create an instance of BCELoss
loss = nn.BCELoss()

# create a tensor with an output probability
p = torch.tensor([.9],dtype=torch.float)

# create a tensor with the actual classification
y = torch.tensor([1],dtype=torch.float)

# compute the BCELoss
print(loss(p,y))
```

Copy to Clipboard

A couple important points:

- we have to create an instance of `nn.BCELoss` instead of calling it directly
- `nn.BCELoss` takes two tensors as input: the probability first and the actual classification second

In the code cell below, use `nn.BCELoss` to compute the loss for:

- probability `.7`
- actual classification `0`

Save your computed loss to the variable `loss_value`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

The checkpoint instructions contain code for `p=.9` and `y=1`. Make the changes needed to calculate `p=.7` and `y=0`.

Checkpoint 3 Passed

3.

What happens if the model outputs `.5` – an even 50/50 chance of the input being classified as `1` or `0`.

Use PyTorch's BCELoss to compute BCELoss for `p=.5` in the cell below, for both `y=1` and `y=0`.

Save the result for `y=1` to the variable `loss1` and the result for `y=0` to the variable `loss0`.

What do you notice about the results?

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Take the code from your Checkpoint 2 solution and modify it to calculate:

- `p=.5` and `y=1`
- `p=.5` and `y=0`

Example - Binary Cross-Entropy (BCE) Loss Function

In the code cell below, we use Numpy's `np.log` function to implement BCELoss and compute the examples from the narrative.

Throughout this notebook, we use `y` to stand for the true classification, since it is the target value of classification.

Run the code cell, and compare to the example in the narrative. Feel free to play around in this cell, changing the `p` and `y` values to get a feel for how BCELoss works!

```
# import NumPy
import numpy as np

# define BCELoss for a probability p and a true classification y
def BCELoss(p,y):
   if y == 1: #if the true classification is 1
       return -np.log(p)
   else: # if the true classification is 0
       return -np.log(1-p)

# compute and print examples from the narrative
```

```
p = .9
print("Actual classification 1")
print("When p is " + str(.9) + ", the loss is " + str(BCELoss(p,1)))
print("Actual classification 0")
print("When p is " + str(.9) + ", the loss is " + str(BCELoss(p,0)))
```
Actual classification 1
When p is 0.9, the loss is 0.10536051565782628
Actual classification 0
When p is 0.9, the loss is 2.302585092994046

Checkpoint 1/3
Suppose a network outputs a probability `p = .85`. If the BCELoss for this prediction is fairly low, what is the true classification?
Uncomment the line in the next cell corresponding to the answer to this question.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
```
## YOUR SOLUTION HERE ##
y = 1
#y = 0
```
Checkpoint 2/3
PyTorch implements BCELoss in `torch.nn.BCELoss()`. Here's an example:
```
import torch
from torch import nn

# create an instance of BCELoss
loss = nn.BCELoss()

# create a tensor with an output probability
p = torch.tensor([.9],dtype=torch.float)

# create a tensor with the actual classification
y = torch.tensor([1],dtype=torch.float)

# compute the BCELoss
    print(loss(p,y))
```
A couple important points:

- we have to create an instance of `nn.BCELoss` instead of calling it directly
- `nn.BCELoss` takes two tensors as input: the probability first and the actual classification second

In the code cell below, use `nn.BCELoss` to compute the loss for:

- probability `.7`
- actual classification `0`

Save your computed loss to the variable `loss_value`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
```
import torch
from torch import nn

## YOUR SOLUTION HERE ##

# create an instance of BCELoss
loss = nn.BCELoss()

# create a tensor with an output probability
p = torch.tensor([.7],dtype=torch.float)

# create a tensor with the actual classification
y = torch.tensor([0],dtype=torch.float)

# define loss_value
loss_value = loss(p,y)

# print the BCELoss
loss_value
```
tensor(1.2040)
Checkpoint 3/3
What happens if the model outputs `.5` -- an even 50/50 chance of the input being classified as `1` or `0`.
Use PyTorch's BCELoss to compute BCELoss for `p=.5` in the cell below, for both `y=1` and `y=0`.
Save the result for `y=1` to the variable `loss1` and the result for `y=0` to the variable `loss0`.
What do you notice about the results?
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
```
import torch
```

```
from torch import nn

# create an instance of BCELoss
loss = nn.BCELoss()


## YOUR SOLUTION HERE ##

p = torch.tensor([.5],dtype=torch.float)

# compute loss for p=.5 and y=1 here
y1 = torch.tensor([1],dtype=torch.float)
loss1 = loss(p,y1)

# compute loss for p=.5 and y=0 here
y0 = torch.tensor([0],dtype=torch.float)
loss0 = loss(p,y0)

print("Loss for y=1: " + str(loss1))
print("Loss for y=0: " + str(loss0))
```

Loss for y=1: tensor(0.6931)
Loss for y=0: tensor(0.6931)

Because our model is completely uncertain about the classification (pure 50/50 guess!) the loss is actually the same for each possible true classification!

Training

It's time to train a classification model! Because this course assumes some prior familiarity with training neural networks, we won't cover every step in detail. But don't worry, we've included sample code here and in the notebook so you don't have to remember specific syntax.

That said, let's quickly review the basic steps of training a neural network:

- Split the dataset into training and testing sets
- Initialize a **loss function**
- Initialize an **optimizer**, and select a **learning rate**
- Select a number of epochs (the number of training iterations through the network)
- Run the training loop for that number of epochs

In this exercise, we'll use the `nn.BCELoss()` loss function we've already covered.

For the optimizer, we'll use **stochastic gradient descent (SGD)**. SGD is a variant of the traditional gradient descent. Instead of using the entire training set to compute the gradient of the loss function, SGD uses a single randomly chosen data point.

The syntax to initialize SGD is

```
optimizer = optim.SGD(
    model.parameters(),
    lr=0.001)
```

```
```

Copy to Clipboard

where

- `model` is our neural network
- `lr` is the learning rate, which controls how much the weights and biases are updated in each training iteration

Accuracy

The training loop for classification is basically the same as for regression. One difference is that, on top of tracking loss, we'll also track **accuracy** .

Remember, our loss function is based on the output probability from the network. But we also care about the accuracy of the final binary predictions, after applying a threshold to the output sigmoid probabilities. This is where **accuracy** comes in:

$$\text{Accuracy} = \frac{\text{\# of correct predictions}}{\text{\# of predictions}}$$

For example, suppose our model correctly predicted pass/fail outcome for 72 students, and made the wrong prediction for the remainder. This would result in an accuracy of

$$\frac{72}{100} = 0.72 \text{ (or 72\%)}$$

In our code, we can use the function `accuracy_score` from scikit-learn:

```
from sklearn.metrics import accuracy_score



# use a threshold of .5 to predict classifications

predicted_labels = (predictions >= 0.5).int()
```

```
# calculate accuracy

accuracy = accuracy_score(y_train, predicted_labels)
```

Copy to Clipboard

Training Loop

As a refresher, here's a full example of a training loop:

```
loss = nn.BCELoss()

optimizer = optim.SGD(

    nn_classifier.parameters(),

    lr=0.001)



num_epochs = 500

for epoch in range(num_epochs):

    # pass the training data through the model, producing sigmoid
probabilities

    predictions = model(X_train)



    # calculate the loss between predictions and the true
classifications

    BCELoss = loss(predictions, y_train)
```

```
    # perform the backward pass to compute gradients
    BCELoss.backward()


    # use SGD to update weights and biases
    optimizer.step()


    # reset the gradients for the next iteration
    optimizer.zero_grad()
```

Copy to Clipboard

Within the loop, we'll also want to print out both loss and accuracy at regular intervals. This will help us evaluate how effective our training process is.

Here's sample code to print loss and accuracy every 100 epochs:

```
# if the epoch number is 100, 200, 300, ...
if (epoch + 1) % 100 == 0:
        # use a threshold of .5 to transform sigmoid probabilities
into classifications
        predicted_labels = (predictions >= 0.5).int()


        # compute the accuracy
        accuracy = accuracy_score(y_train, predicted_labels)
```

```
        # print the BCELoss and accuracy

        print(f'Epoch [{epoch+1}/{num_epochs}], BCELoss:
{BCELoss.item():.4f}, Accuracy: {accuracy.item():.4f}')
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

First, run the **Setup**, **Import Data**, and **Create the Training and Testing Sets**. Don't modify these, as that will impact our ability to test your code later.

Now, suppose we trained a model on 5 students (a bit silly, but stay with us here.)

Of those five students, our model predicted:

Student 1 passed (true classification: failed)

Student 2 passed (true classification: passed)

Student 3 failed (true classification: failed)

Student 4 passed (true classification: failed)

Student 5 failed (true classification: passed)

Calculate the accuracy of this set of predictions. Assign the result to the variable `accuracy`.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Count the number of correct predictions, and divide that by the total number of predictions made.

Checkpoint 2 Passed

2.

We've already created a neural network for classification, named `model`.

Fill in the rest of the code, following our commented instructions.

The code to keep track of the accuracy and loss during training is also provided. Do not modify this, as we'll test your work by evaluating the output of the training loop.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Use our sample code from the narrative to guide you – most of it will be the same!

Checkpoint 3 Passed

3.

What did you notice about the BCE loss and accuracy during training? The BCE loss decreased little by little which is a good sign, but the accuracy remained the same which is not ideal.

One solution is to change the **learning rate** of our SGD optimizer.

Re-do the training from Checkpoint 1 (just copy and paste the code), but modify the learning rate to `.01`.

Does the accuracy increase during training? What about the BCE loss?

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Start by copying and pasting the code from Checkpoint 2 into the Checkpoint 3 code cell. Then find where the optimizer is initialized, and modify the learning rate.

Checkpoint 4 Passed

4.

It looks like the BCE loss is decreasing much more than before which is good! However, the accuracy remains the same.

Another solution we can try is to increase the number of training epochs.

Let's keep the learning rate at `0.01` and **increase** the number of epochs to `1000` and re-run the training loop.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Copy and paste the code from Checkpoint 3 into the Checkpoint 4 code cell. Then, find where the number of epochs is defined, and modify it to 1000.

Setup
Run the next cell to import all the libraries we'll need.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
```

Import Data
The code cell below imports the encoded student performance dataset and creates two DataFrames:

- `X` contains all the input features we will train our model on
- `y` contains the targets, the true classifications (pass or fail)

```
# Load the dataset
df = pd.read_csv("student performances encoded.csv")

# Remove columns from training:
# - Student ID since it is just a unique row identifier
# - Letter Grade since that contains info that directly determines the target column
# - Outcome since that is the target column

remove_cols = ['Student ID', 'Letter Grade', 'Outcome']
train_features = [x for x in df.columns if x not in remove_cols]

# Create tensor of input features
X = torch.tensor(df[train_features].values, dtype=torch.float)
# Create tensor of targets
y = torch.tensor(df['Outcome'].values, dtype=torch.float).view(-1,1)

df.head()
```

5 rows × 58 columns

Create the Training and Testing Sets
The code cell below splits `X` and `y` into training and testing sets using scikit-learn's `train_test_split`.
Do not modify our proportions or random state, as that will impact code testing later on.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    train_size=0.80, # use 80% of the data for training
                                                    test_size=0.20, # use 20% of the data for testing
                                                    random_state=42) # set a random state

print("Training Shape:", X_train.shape)
print("Testing Shape:", X_test.shape)
```

Training Shape: torch.Size([116, 55])
Testing Shape: torch.Size([29, 55])

Checkpoint 1/4
Suppose we trained a model on 5 students (a bit silly, but stay with us here.)
Of those five students, our model predicted:

- Student 1 passed (true classification: failed)
- Student 2 passed (true classification: passed)
- Student 3 failed (true classification: failed)
- Student 4 passed (true classification: failed)
- Student 5 failed (true classification: passed)

Calculate the accuracy of this set of predictions. Assign the result to the variable `accuracy`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for

more details.

```
correct = 2
total = 5
accuracy = correct/total

# show output
print(accuracy)
```

0.4

Checkpoint 2/4

We've already created a neural network for classification, named `model`.
Fill in the rest of the code, following our commented instructions.
The code to keep track of the accuracy and loss during training is also provided. Do not modify this, as we'll test your work by evaluating the output of the training loop.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# Set a random seed - do not modify
torch.manual_seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(55, 110), # 55 is the number of input features in X_train
    nn.ReLU(),
    nn.Linear(110, 55),
    nn.ReLU(),
    nn.Linear(55, 1), # one output node for binary classification
    nn.Sigmoid() # sigmoid activation to output probabilities
)

## YOUR SOLUTION HERE ##

# Import accuracy_score function from sklearn.metrics
from sklearn.metrics import accuracy_score

# initialize the BCE loss function
loss = nn.BCELoss()

# initialize SGD optimizer, with a learning rate of .001
optimizer = optim.SGD(model.parameters(), lr=0.001)

# set the number of epochs to 300
num_epochs = 300

for epoch in range(num_epochs):
    ## Add forward pass here, keep the variable name predictions ##
    predictions = model(X_train)

    ## Compute BCELoss loss here ##
    BCELoss = loss(predictions, y_train)

    ## Compute gradients here ##
    BCELoss.backward()

    ## Update weights and biases here ##
    optimizer.step()

    ## Reset the gradients for the next iteration here ##
    optimizer.zero_grad()

    ## DO NOT MODIFY ##
    # keep track of the accuracy and loss during training
    if (epoch + 1) % 100 == 0:
        predicted_labels = (predictions >= 0.5).int()
        accuracy = accuracy_score(y_train, predicted_labels)
        print(f'Epoch [{epoch+1}/{num_epochs}], BCELoss: {BCELoss.item():.4f}, Accuracy: {accuracy.item():.4f}')
```

Epoch [100/300], BCELoss: 0.6844, Accuracy: 0.6207
Epoch [200/300], BCELoss: 0.6692, Accuracy: 0.6897
Epoch [300/300], BCELoss: 0.6568, Accuracy: 0.6897

Checkpoint 3/4

What did you notice about the BCE loss and accuracy during training? The BCE loss decreased each time we printed it, which is a good sign, but the accuracy remained the same for the last two.
One solution is to change the **learning rate** of our SGD optimizer.
Re-do the training from Checkpoint 1 (just copy and paste the code), but modify the learning rate to `.01`.
Does the accuracy increase during training? What about the BCE loss?
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# Set a random seed - do not modify
torch.manual_seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(55, 110),
    nn.ReLU(),
    nn.Linear(110, 55),
    nn.ReLU(),
    nn.Linear(55, 1),
    nn.Sigmoid()
)

## YOUR SOLUTION HERE ##

# Import accuracy_score function from sklearn.metrics
from sklearn.metrics import accuracy_score

# initialize the BCE loss function
loss = nn.BCELoss()
```

```
# initialize SGD optimizer
optimizer = optim.SGD(model.parameters(), lr=0.01)

# set the number of epochs to 300
num epochs = 300

for epoch in range(num epochs):
    ## Add forward pass here, keep the variable name predictions ##
    predictions = model(X train)

    ## Compute BCELoss loss here ##
    BCELoss = loss(predictions, y train)

    ## Compute gradients here ##
    BCELoss.backward()

    ## Update weights and biases here ##
    optimizer.step()

    ## Reset the gradients for the next iteration here ##
    optimizer.zero grad()

    ## DO NOT MODIFY ##
    # keep track of the accuracy and loss during training
    if (epoch + 1) % 100 == 0:
        predicted_labels = (predictions >= 0.5).int()
        accuracy = accuracy score(y train, predicted labels)
        print(f'Epoch [{epoch+1}/{num epochs}], BCELoss: {BCELoss.item():.4f}, Accuracy: {accuracy.item():.4f}'
```

Epoch [100/300], BCELoss: 0.6054, Accuracy: 0.6897
Epoch [200/300], BCELoss: 0.5853, Accuracy: 0.6897
Epoch [300/300], BCELoss: 0.5722, Accuracy: 0.6897

Checkpoint 4/4
It looks like the BCE loss is decreasing much more than before which is good! However, the accuracy remains the same.
Another solution we can try is to increase the number of training epochs.
Let's keep the learning rate at `0.01` and **increase** the number of epochs to `1000` and re-run the training loop.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# Set a random seed - do not modify
torch.manual seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(55, 110),
    nn.ReLU(),
    nn.Linear(110, 55),
    nn.ReLU(),
    nn.Linear(55, 1),
    nn.Sigmoid()
)

## YOUR SOLUTION HERE ##

# Import accuracy score function from sklearn.metrics
from sklearn.metrics import accuracy score

# initialize the BCE loss function
loss = nn.BCELoss()

# initialize SGD optimizer, with a learning rate of .001
optimizer = optim.SGD(model.parameters(), lr=0.01)

# set the number of epochs
num epochs = 1000

for epoch in range(num epochs):
    ## Add forward pass here, keep the variable name predictions ##
    predictions = model(X train)

    ## Compute BCELoss loss here ##
    BCELoss = loss(predictions, y train)

    ## Compute gradients here ##
    BCELoss.backward()

    ## Update weights and biases here ##
    optimizer.step()

    ## Reset the gradients for the next iteration here ##
    optimizer.zero grad()

    ## DO NOT MODIFY ##
    # keep track of the accuracy and loss during training
    if (epoch + 1) % 100 == 0:
        predicted labels = (predictions >= 0.5).int()
        accuracy = accuracy score(y train, predicted labels)
        print(f'Epoch [{epoch+1}/{num epochs}], BCELoss: {BCELoss.item():.4f}, Accuracy: {accuracy.item():.4f}'
```

Epoch [100/1000], BCELoss: 0.6054, Accuracy: 0.6897
Epoch [200/1000], BCELoss: 0.5853, Accuracy: 0.6897
Epoch [300/1000], BCELoss: 0.5722, Accuracy: 0.6897
Epoch [400/1000], BCELoss: 0.5572, Accuracy: 0.6897
Epoch [500/1000], BCELoss: 0.5403, Accuracy: 0.6897
Epoch [600/1000], BCELoss: 0.5217, Accuracy: 0.7241
Epoch [700/1000], BCELoss: 0.5017, Accuracy: 0.7328
Epoch [800/1000], BCELoss: 0.4808, Accuracy: 0.7586
Epoch [900/1000], BCELoss: 0.4595, Accuracy: 0.7586

Epoch [1000/1000], BCELoss: 0.4376, Accuracy: 0.7672

Nice! It looks like during training, the model's BCE loss is decreasing (good) and the accuracy is increasing (also good)!
In the next exercise, we'll evaluate our model on the unseen testing set by computing the accuracy and other classification metrics.

Evaluation

After training, we need to evaluate our model on our testing dataset.

We can separate model outputs into four main groups:

- **false positive (FP)**: the model predicts `1` but the true label is `0`
- **false negative (FN)**: the model predics `0` but the true label is `1`
- **true positive (TP)**: the model predicts `1` and the true label is `1`
- **true negative (TN)**: the model predicts `0` and the true label is `0`

The **accuracy**, in this context, is

$$\text{Correct predictions} / \text{All predictions} = (TP+TN) / (TP+TN+FP+FN)$$

For example, suppose we have 10 students, and the model gives

- 8 TNs (students who fail, and our model outputs `0`)
- 1 TP (student passed, model outputs `1`)
- 1 FP (student failed, model outputs `0`)

We've only got one incorrect prediction (the FP), resulting in an accuracy of

$$\text{Correct predictions} / \text{All predictions} = (8+1) / 10 = 90\%$$

That's pretty good!

But what if the goal of our model is to identify failing students, so that we can give them more help? In this case, the false positive is a real concern. This student failed, but our model did not warn us ahead of time.

Let's explore some other evaluation metrics, also called evaluation scores.

Precision

Precision emphasizes false positives in the denominator:

$$\text{Precision} = TP / (TP+FP)$$

In our example, there is 1 TP and 1 FP, so we get

$$\text{Precision} = 1 / (1+1) = .5$$

Precision being 50% means that half of our model's positive predictions were false.

Using precision, instead of accuracy, would alert us immediately to the false positive and encourage us to improve our model's performance!

Recall

Unlike precision, recall emphasizes false *negatives*:

$$\text{Recall} = TP / (TP+FN)$$

In our case, there was 1 true positive and 0 false negatives, so our recall is

$$1 / (1+0) = 1 = 100\%$$

F1 Score

The **F1 score** is the harmonic mean of precision and recall, which means it balances concern for false positives and false negatives:

$$F1 = (2 * \text{Precision} * \text{Recal}) / (\text{lPrecision}+\text{Recall})$$

While this formula is a bit complex, the rule of thumb is that

higher false positives will result in lower precision values

higher false negatives will result in lower recall values

the F1 score balances between the precision and recall values

In our case, precision was `.5` and recall was `1`, so the F1-score is

$$(2 * .5 * 1) / (.5+1) = 1 / 1.5 = .67$$

F1 is often a good first evaluation metric to try.

Scikit-Learn Code

Scikit-Learn provides `precision_score`, `recall_score`, and `f1_score` functions just like the `accuracy_score` function we used in the last exercise.

```
from sklearn.metrics import accuracy score, precision score,
recall_score, f1_score


accuracy = accuracy_score(y_test, predicted_labels)
precision = precision_score(y_test, predicted_labels)
recall = recall_score(y_test, predicted_labels)
f1 = f1_score(y_test, predicted_labels)
```

Copy to Clipboard

Instructions

Checkpoint 1 Passed

1.

Suppose we are building a model to detect a rare disease in patient scans:

positive class / label `1` is patients with the disease

negative class / label `0` is patients without the disease

In this case, suppose we want to be as certain as possible that there are no false negatives. That is, we don't want any patients with the disease to be incorrectly classified as healthy.
Which evaluation metric would best reflect this priority? Uncomment the corresponding line in the cell below.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Which metric emphasizes false negatives?

Checkpoint 2 Passed

2.

First, run the cells to import the data, perform a train/test split, and train the model.
Generate predicted probabilities from `model` on the testing set `X_test`. Save them to the variable `test_predictions`.
Then, use `test_predictions` to create predicted labels (`1`/`0`) using a threshold of `0.5`. Save the predicted labels, as integers, to the variable `test_predicted_labels`.
We've already set the model to evaluation mode using `model.eval()` and turned off the gradient calculations using `with torch.no_grad()`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

To produce predicted probabilities, input `X_test` to `model()`.
To compute predicted labels, begin by determining which probabilities are above the threshold:

```
test_predictions >= .5
```

Copy to Clipboard

Then, convert the resulting array to integers.

Checkpoint 3 Passed

3.

Finally, let's evaluate our trained neural network on the testing set by computing the accuracy, precision, recall, and F1 score. We'll use the predicted labels `test_predicted_labels` from the last checkpoint and the actual labels stored in `y_test`.
Save the test scores to the following variables:

`test_accuracy` contains the accuracy score

`test_precision` contains the precision score

`test_recall` contains the recall score

`test_f1` contains the F1 score

How does our trained neural network perform?
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

For each score, input `y_test` and `test_predicted_labels` to the evaluation function from scikit-learn.

Setup
Run the setup cell to import all the libraries we'll need for this exercise.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
```

Example - Calculate Accuracy, Precision, Recall, and F1 Score
Run the next code cell to calculate all four evaluation metrics on the example from the narrative. Feel free to experiment with different sets of predictions! How do the metrics change if the model's predictions change?

```
from sklearn.metrics import accuracy score, precision score, recall score, f1 score

# Actual results: first nine students failed, last one passed
y true = [0] * 9 + [1] * 1
# Model predictions: first 8 students failed, last two passed (producing one false positive)
y pred = [0]*8 + [1]*2

# Print y true and y pred
print("y true:", y true)
print("y pred:", y pred)

# Calculate classification metrics
accuracy = accuracy score(y true, y pred)
precision = precision score(y true, y pred)
recall = recall score(y true, y pred)
f1 = f1 score(y true, y pred)

# Print the results
print("Accuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("F1 Score:", f1)
```

y_true: [0, 0, 0, 0, 0, 0, 0, 0, 0, 1]
y_pred: [0, 0, 0, 0, 0, 0, 0, 0, 1, 1]
Accuracy: 0.9
Precision: 0.5
Recall: 1.0
F1 Score: 0.6666666666666666

Checkpoint 1/3
Suppose we are building a model to detect a rare disease in patient scans:

- **positive class / label `1`** is patients with the disease
- **negative class / label `0`** is patients without the disease

In this case, suppose we want to be as certain as possible that there are no false negatives. That is, we don't want any patients with the disease to be incorrectly classified as healthy.
Which evaluation metric would best reflect this priority? Uncomment the corresponding line in the cell below.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
## YOUR SOLUTION HERE ##
```

```
#metric = "accuracy"
#metric = "precision"
metric = "recall"
#metric = "f1"
```

Import Data

Let's get ready to evaluate our student success model.

First, run the next code cell to create the training feature dataset X and the labels y.

Note: do not modify this code as it will impact our code testing later on!

```
# Load the dataset
df = pd.read_csv("student_performances_encoded.csv")

# Select training features
remove_cols = ['Student ID', 'Letter Grade', 'Outcome']
train_features = [x for x in df.columns if x not in remove_cols]

# Create tensor of input features
X = torch.tensor(df[train_features].values, dtype=torch.float)
# Create tensor of targets
y = torch.tensor(df['Outcome'].values, dtype=torch.float).view(-1,1)

# Preview the dataset
df.head()
```

5 rows × 58 columns

Create the Training and Testing Sets

Run the next code cell to split X and y into training and testing dataset.s

Note: do not modify this code as it will impact our code testing later on!

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    train_size=0.80, # use 80% of the data for tra
                                                    test_size=0.20, # use 20% of the data for test
                                                    random_state=42) # set a random state
```

Train the Neural Network

Run the next cell to train the network.

Note: do not modify this code as it will impact our code testing later on!

```
# Set a random seed
torch.manual_seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(55, 110),
    nn.ReLU(),
    nn.Linear(110, 55),
    nn.ReLU(),
    nn.Linear(55, 1),
    nn.Sigmoid()
)

# BCE loss function + SGD optimizer
loss = nn.BCELoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)

# Train the neural network
num_epochs = 1000
for epoch in range(num_epochs):
    predictions = model(X_train)
    BCELoss = loss(predictions, y_train)
    BCELoss.backward()
    optimizer.step()
    optimizer.zero_grad()

    # keep track of the accuracy and loss during training
    if (epoch + 1) % 100 == 0:
        predicted_labels = (predictions >= 0.5).int()
        accuracy = accuracy_score(y_train, predicted_labels)
        print(f'Epoch [{epoch+1}/{num_epochs}], BCELoss: {BCELoss.item():.4f}, Accuracy:
{accuracy.item():.4f}')
```

```
Epoch [100/1000], BCELoss: 0.6054, Accuracy: 0.6897
Epoch [200/1000], BCELoss: 0.5853, Accuracy: 0.6897
Epoch [300/1000], BCELoss: 0.5722, Accuracy: 0.6897
Epoch [400/1000], BCELoss: 0.5572, Accuracy: 0.6897
Epoch [500/1000], BCELoss: 0.5403, Accuracy: 0.6897
Epoch [600/1000], BCELoss: 0.5217, Accuracy: 0.7241
Epoch [700/1000], BCELoss: 0.5017, Accuracy: 0.7328
Epoch [800/1000], BCELoss: 0.4808, Accuracy: 0.7586
```

Epoch [900/1000], BCELoss: 0.4595, Accuracy: 0.7586
Epoch [1000/1000], BCELoss: 0.4376, Accuracy: 0.7672

Checkpoint 2/3
Generate predicted probabilities from `model` on the testing set `X_test`. Save them to the variable `test_predictions`.
Then, use `test_predictions` to create predicted labels (`1`/`0`) using a threshold of `0.5`. Save the predicted labels, as integers, to the variable `test_predicted_labels`.
We've already set the model to evaluation mode using `model.eval()` and turned off the gradient calculations using `with torch.no_grad()`.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
model.eval()
with torch.no_grad():
    ## YOUR SOLUTION HERE ##
    test_predictions = model(X_test)
    test_predicted_labels = (test_predictions >= 0.5).int()

# show output - do not remove or modify
print(test_predicted_labels)
```

```
tensor([[1],
        [0],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1],
        [0],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1],
        [0],
        [1],
        [1],
        [1],
        [1],
        [1],
        [1]], dtype=torch.int32)
```

Checkpoint 3/3
Finally, let's evaluate our trained neural network on the testing set by computing the accuracy, precision, recall, and F1 score.
We'll use the predicted labels `test_predicted_labels` from the last checkpoint and the actual labels stored in `y_test`.
Save the test scores to the following variables:

- `test_accuracy` contains the accuracy score
- `test_precision` contains the precision score
- `test_recall` contains the recall score
- `test_f1` contains the F1 score

How does our trained neural network perform?
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

## YOUR SOLUTION HERE ##
test_accuracy = accuracy_score(y_test, test_predicted_labels)
test_precision = precision_score(y_test, test_predicted_labels)
test_recall = recall_score(y_test, test_predicted_labels)
test_f1 = f1_score(y_test, test_predicted_labels)

# show output - do not remove or modify
print("Accuracy:", test_accuracy)
print("Precision:", test_precision)
print("Recall:", test_recall)
print("F1 Score:", test_f1)
```

```
Accuracy: 0.7931034482758621
Precision: 0.8076923076923077
Recall: 0.9545454545454546
F1 Score: 0.875
```

Nice! Our precision is 80%, meaning that 80% of the students we said would pass did indeed pass. The more balanced F1-score is .875.
Now, that still leaves room for improvement, but we've started with a pretty basic model. Once you pass this exercise, feel free to experiment with the model design and training to try to improve our result!

Multiclass Models

So far, we've been working on a **binary classification** problem. Our model predicts one of two outputs: the student will pass (output `1`) or fail (output `0`).
But what about **multiclass classification**, classification problems with more than two possible output categories?
For example, suppose we wanted to classify students into three categories by where they live:

lives in the on-campus **dormitories**
lives with **family** off-campus
lives **independently off-campus** (i.e. off-campus and not with family)

Our strategy for binary classification won't work in this case, since our binary classification networks output probabilities between `0` and `1`, which aren't easily translated into more than two labels.
Instead, we'll use one output node for each possible label:

dorm output node: probability of living in the dorm (`1`) or not (`0`)
family output node: probability of living with family (`1`) or not (`0`)
independent output node: probability of living independently off campus (`1`) or not (`0`)

Note the similarity to one-hot encoding! Here, each label receives its own binary output node, just like in one-hot encoding each category receives its own binary column.
Like one-hot encoding, this setup also assumes that the labels are independent from each other. If there is overlap between the labels (a student that is `1` for multiple labels), this strategy won't work.
Instructions

Checkpoint 1 Failed, try again

1.

Suppose we wanted to predict whether a student:

always takes notes
almost always takes notes
sometimes takes notes
never takes notes

We've started building a neural network to model this problem. Add a linear output layer with the right number of output nodes to the `model` we started.
Do not include any activation functions after the output layer.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.
How many labels are we trying to predict? How will each those be represented in the output layer?

Checkpoint 1/1
Suppose we wanted to predict whether a student:

- always takes notes
- almost always takes notes
- sometimes takes notes
- never takes notes

We've started building a neural network to model this problem. Add a linear output layer with the right number of nodes to the `model` we started.
Do not include any activation functions after the output layer.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
import torch
from torch import nn

# Set a random seed - do not modify
torch.manual_seed(42)

# Define the model using nn.Sequential
model = nn.Sequential(
    nn.Linear(5, 110),
    nn.ReLU(),
    nn.Linear(110, 55),
    nn.ReLU(),
    ## YOUR SOLUTION HERE ##
    nn.Linear(55,4))
```

Multiclass: softmax

https://www.codecademy.com/courses/py-torch-for-classification/lessons/py-torch-classification/exercises/multiclass-softmax

In our last exercise, we were exploring a multiclass model with three labels. We had one output node for each possible label:

dorm output node: probability of living in the dorm (1) or not (0)
family output node: probability of living with family (1) or not (0)
independent output node: probability of living independently off campus (1) or not (0)

In binary classification, we'd use sigmoid activation to produce probabilities between `0` and `1` for an output node.
Using sigmoid on its own won't work as well for multiclass, because sigmoid operates independently on each node.
As an example, here's what sigmoid output could look like:

dorm output: .9
family output: .8
independent output: .4

Interpreted as probabilities, this output means that our model predicts a 90% probability the student lives in the dorm and an 80% probability that the student lives at home with their family.
But this doesn't totally make sense: if there's a 90% chance of living in the dorm, how is there also an 80% chance of living at home with family? Because sigmoid operates on each node independently, the output isn't consistent.

Softmax Function

We'll use a function called `softmax` to normalize these outputs so that they form a unified probability distribution.
For example, after applying softmax the sigmoid outputs above become

dorm output: .4
family output: .36
independent off-campus output: .24

All three probabilities now sum to `1`, or `100%`, which is much easier to interpret. We can now say that our model predicts a 40% probability of living in the dorms, 36% probability of living with family, and 24% probability of living off-campus.

Softmax in PyTorch

PyTorch does offer an implementation of softmax in `nn.Softmax()`. However, PyTorch automatically applies a version of softmax in its cross entropy loss algorithm for multiclass networks, so we won't need to use it ourselves.
While PyTorch builds it in for ease, it is helpful to understand what softmax is doing for a couple reasons:

in some advanced cases, it may not be built in
when it is built in, we should still be aware that the transformation is happening

In the notebook, we'll guide you through computing some actual examples of softmax!

Instructions

Checkpoint 1 Passed

1.

Let's dig into how softmax actually works.
Suppose we are starting with the network outputs of `[.9, .8, .4]`.
We begin by calculating a **normalization factor**. This is a fancy term for something we'll use to make sure our outputs sum to `1`. For our example with softmax, the normalization factor is

$$e^{.9}+e^{.8}+e^{.4}$$

Basically, we

take each output
apply the exponential function
add up the exponential outputs

Then, we take each of our individual outputs and divide by the normalization factor.
For `.9`, our softmax output will be

$$\frac{e^{.9}}{e^{.9}+e^{.8}+e^{.4}}$$

In the following cell, we've calculated the softmax output for `.9` in this array.
Calculate the softmax output for `.8` and assign it to `softmax_8`.
Note that

```
np.exp(x)
```

```
Copy to Clipboard
```

is equivalent to

ex

$$e$$

x

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Compute `np.exp(.8)` and then divide the result by the normalization factor.

Checkpoint 2 Passed

2.

Is it possible for the array

```
[.6, .4, .5]
```

```
Copy to Clipboard
```

to be the output of softmax? Uncomment the correct answer in the next cell.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

The goal of softmax is to produce a probability distribution. What does that mean about the set of numbers softmax can output?

Checkpoint 1/2

Let's dig into how softmax actually works.

Suppose we are starting with the network outputs of `[.9, .8, .4]`.

We begin by calculating a **normalization factor**. This is a fancy term for something we'll use to make sure our outputs sum to `1`. For our example with softmax, the normalization factor is

$$e^{.9} + e^{.8} + e^{.4}$$

Basically, we

- take each output
- apply the exponential function
- add up the exponential outputs

Then, we take each of our individual outputs and divide by the normalization factor.

For `.9`, our softmax output will be

$$\frac{e^{.9}}{e^{.9} + e^{.8} + e^{.4}}$$

In the following cell, we've calculated the softmax output for `.9` in this array.

Calculate the softmax output for `.8` and assign it to `softmax_8`.

Note that

np.exp(x)

is equivalent to

$$e^x$$

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
import numpy as np

softmax_9 = np.exp(.9) / (np.exp(.9) + np.exp(.8) + np.exp(.4))

## YOUR SOLUTION HERE ##

softmax_8 = np.exp(.8) / (np.exp(.9) + np.exp(.8) + np.exp(.4))

# show output
print(np.round(softmax 9,2))
print(np.round(softmax_8,2))
```

0.4

0.36

Checkpoint 2/2

Is it possible for the array

[.6, .4, .5]

to be the output of softmax? Uncomment the correct answer in the next cell.

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
#answer = "yes"
answer = "no"

# show output
answer
```

'no'

Explanation

Softmax outputs a probability distribution. The array in question sums to `.6 + .4 + .5 = 1.5`, while a probability distribution has to sum to `1`.

https://www.codecademy.com/courses/py-torch-for-classification/lessons/py-torch-classification/exercises/multiclass-argmax

Multiclass: argmax

So how do we determine the final label predictions in a multiclass problem?

Let's return to the example of building a model to predict if a student

- always takes notes
- almost always takes notes
- sometimes takes notes
- never takes notes

Because this data needs to be encoded, let's assume we've label encoded the column as

- 0: always takes notes
- 1: almost always takes notes
- 2: sometimes takes notes
- 3: never takes notes

Suppose we feed two students' data through our network. Our output will be a tensor with two rows, one corresponding to each student. Something like this:

```
[[-0.1480,  0.0144, -0.0396,  0.0027],
[-0.1065, -0.0680,  0.0191,  0.0787]]
```

Copy to Clipboard

To make this easier to interpret, we could apply softmax to produce probabilities:

```
[[0.2246, 0.2641, 0.2502, 0.2611],
[0.2285, 0.2375, 0.2591, 0.2750]]
```

Copy to Clipboard

Let's interpret this output!

Each row corresponds to a student:

Student 1: in the first row, the largest probability is `.2641`, or `26.41%`. This probability occurs in the second column, which is index `1`. This means our model classifies the first student as label `1`, or **almost always takes notes**.

Student 2: in the second row, the largest probability is `.2750`, or `27.5%`. This probability occurs in the fourth column, which is index `3`. This means our model classifies the second student as label `3`, or **never takes notes**.

argmax

To identify these classifications programmatically, we'd call

```
torch.argmax(softmax_output,dim=1)
```

Copy to Clipboard

where

`softmax_output` contains the output array above

`dim=1` tells argmax to find the largest value in each *row*

The output would be

```
tensor([1, 3])
```

Copy to Clipboard

Notice that these match the labels we got by hand! The first student is classified as `1` (almost always takes notes) and the second student as `3` (never takes notes). Pretty cool!

Do we need softmax?

We applied softmax to make the outputs easier to interpret as probabilities. But `argmax` will produce the same output whether we call `softmax` first or not.

Calling `softmax` is important in training (PyTorch will do this automatically for us) or to interpret the output as a probability distribution (for example, if we want to see "how confident" the final predictions is). But if all we care about is the final classification, it isn't necessary.

Instructions

Checkpoint 1 Passed

1.

We've defined a tensor `raw_output` in the code cell. This tensor contains the output of our neural network labeling students with

0: always takes notes
1: almost always takes notes
2: sometimes takes notes
3: never takes notes

For the first student, define the following variables:

`largest_output`: the numeric value of the largest output for the first student
`largest_output_index`: the index corresponding to the largest output (0, 1, 2, or 3)
`predicted_label`: the numeric label the model predicts for the first student

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

`largest_output`: the first student corresponds to the first row. Which is the largest value? You should assign a 4-decimal number to `largest_output`?

`largest_output_index`: remember that the index starts counting at `0`?

`predicted_label`: what is the **number** representing the label?

Checkpoint 2 Passed

2.

We've defined the same network outputs in this code cell.

Use `argmax` to identify the index of the largest output for all five students. Assign the results to the variable `argmax_output`.

Which student takes the fewest notes?

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

The first input to `argmax` should be the tensor we defined. The second should specify the dimension (`dim=1` will find the index of the largest value in each row).

Setup

Run the cell below to import libraries.

```
import torch
from torch import nn
```

Checkpoint 1/2

We've defined a tensor `raw_output` in the code cell. This tensor contains the output of our neural network labeling students with

- 0: always takes notes
- 1: almost always takes notes
- 2: sometimes takes notes
- 3: never takes notes

For the first student, define the following variables:

- `largest_output`: the numeric value of the largest output for the first student
- `largest_output_index`: the index corresponding to the largest output (0,1,2, or 3)
- `predicted_label`: the numberic label the model predicts for the first student

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for

more details.

```
raw_output = torch.tensor([[0.1320, 0.0160, 0.9614, 0.9919],
       [0.7180, 0.7303, 0.6234, 0.1197],
       [0.8757, 0.2045, 0.1977, 0.3845],
       [0.8934, 0.5677, 0.1377, 0.6420],
       [0.4017, 0.8363, 0.1119, 0.6557]], dtype = torch.float)

labels = {0: 'always takes notes',
1: 'almost always takes notes',
2: 'sometimes takes notes',
3: 'never takes notes'}

## YOUR SOLUTION HERE ##
largest_output = torch.max(raw_output, dim=1)
largest_output_index = torch.argmax(raw_output, dim=1)
predicted_label = [labels[int(i)] for i in largest_output_index]

# show output - do not modify
print("For the first student, the largest output is",largest_output)
print("This corresponds to index",largest_output_index)
print("The predicted label is",predicted_label)
```

For the first student, the largest output is torch.return_types.max(
values=tensor([0.9919, 0.7303, 0.8757, 0.8934, 0.8363]),
indices=tensor([3, 1, 0, 0, 1]))
This corresponds to index tensor([3, 1, 0, 0, 1])
The predicted label is ['never takes notes', 'almost always takes notes', 'always takes notes', 'always takes notes', 'almost always takes notes']

```
int(torch.argmax(raw_output, dim=1)[1])
```

1

Checkpoint 2/2
We've defined the same network outputs in this code cell.

Use `argmax` to identify the index of the largest output for all five students. Assign the results to the variable `argmax_output`.

Which student takes the fewest notes?

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
raw_output = torch.tensor([[0.1320, 0.0160, 0.9614, 0.9919],
       [0.7180, 0.7303, 0.6234, 0.1197],
       [0.8757, 0.2045, 0.1977, 0.3845],
       [0.8934, 0.5677, 0.1377, 0.6420],
       [0.4017, 0.8363, 0.1119, 0.6557]], dtype = torch.float)

## YOUR SOLUTION HERE ##
argmax_output = torch.argmax(raw_output, dim=1)

# show output - do not modify
argmax_output
```

tensor([3, 1, 0, 0, 1])
Looks like the first student is the only one to receive the label 3, corresponding to "never takes notes." We might want to encourage this student to take at least a few more notes!

Multiclass: train and evaluate

Great work! We now have all the pieces we need to train a multiclass network.

Instead of predicting whether students will pass (`1`) or fail (`0`), let's attempt to categorize their final performances as

Below Average: grades 0 and 1

Average: grades 2 and 3

Above Average: grades 4 and 5

We'll create a new column `Performance_Outcome` that labels each student by these three categories. PyTorch assumes the target labels are integers starting at `0`, so we'll use the following labels:

Below Average will be `0`

Average will be `1`

Above Average will be `2`

PyTorch requires target labels to have the `torch.long` datatype, so we'll have to set that datatype when creating our `y` tensor of target labels:

```
y = torch.tensor(df['Performance_Outcome'].values, dtype=torch.long)
```

Copy to Clipboard

Training Setup

As a reminder, these are the steps of the training process:

forward pass: feed the training data through the network

loss: compute the loss between the network output and the training labels

backward pass: calculate the gradients of the loss function

optimizer: apply an optimizer to adjust weights and biases

iterate: reset the gradients, and iterate

We can use the same SGD optimizer from our binary classification model for multiclass classification.

However, the binary cross-entropy loss function is specific to binary problems, so we'll need to use a more general cross-entropy loss function.

Cross-Entropy Loss Function for Multiclass

PyTorch implements a multiclass version of cross-entropy loss in `nn.CrossEntropyLoss()`. As with all other

loss functions

Preview: Docs Loading link description

, we'll start by assigning an instance of `nn.CrossEntropyLoss()` to the variable `loss`:

```
loss = nn.CrossEntropyLoss()
```

Copy to Clipboard

We won't go too deep into the details on `nn.CrossEntropyLoss()`. Essentially, it:

applies a version of softmax to turn the network output into a probability distribution

computes a general version of the negative logarithm loss we covered in detail for binary models

Predict Labels and Evaluation

After training, we feed the test data through the network

```
predictions = model(X_test)
```

Copy to Clipboard

Then we'll create predicted labels using `argmax`:

```
predicted_labels = torch.argmax(predictions, dim=1)
```

Copy to Clipboard

Lastly, just like binary models, we'll compute an evaluation metric. Because we now have multiple classes, the metrics defined in terms of `positive` vs `negative` classes won't work directly.

We can compute accuracy the same way:

```
accuracy = accuracy_score(y_test, predicted_labels)
```

Copy to Clipboard

We can also generate a summary of the classification metrics (precision, recall, and F1 score) for each class using scikit-learn's `classification_report` function:

```
report = classification_report(y_test, predicted_labels)
```

Copy to Clipboard

Since there are more than two classes, we have additional average scores for each metric:

the macro average gives equal weight to each class (arithmetic mean)

the weighted average weights classes with a larger # of observations (their support) higher taking into account class imbalances

We'll go into interpreting the classification report more in the notebook!

Instructions

Checkpoint 1 Passed

1.

Run the **Setup** and **Import and Prepare Data** cells first.
We've created a list of columns to use as input features for our model.
Create a tensor `X` from the columns in `train_features`.
Create a tensor `y` from the column `Performance_Outcome` that contains the target labels.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Here's skeleton code for converting columns of a DataFrame to a tensor:

```
torch.tensor(df[list_of_columns].values, dtype=torch.float
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Make sure to run the **Create the Training and Testing Sets** cell.
Now, let's train a multiclass neural network!
First, instantiate the loss function and optimizer:

instantiate PyTorch's `nn.CrossEntropyLoss()` module and save it to the variable `loss`

assign the **stochastic gradient descent optimizer** with a learning rate of `0.01` to the variable `optimizer`

Next, create the training loop:

train the network for `1000` epochs on the training set `X_train`

calculate the loss using `y_train` and save the loss to the variable `CELoss`

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Make sure to follow our variable names and leave our output code as-is, as those are used in the testing code.
Remember the steps of the training loop:

forward pass: feed the training data through the network

loss: compute the loss between the network output and the training labels

loss.backward(): calculate the gradients of the loss function

optimizer.step(): apply an optimizer to adjust weights and biases

optimizer.zero_grad(): reset the gradients, and iterate

Checkpoint 3 Passed

3.

Let's evaluate our trained neural network on the testing set.
We've already set the model to evaluation mode and turned off the gradient calculations.
Add code to:

Feed the test dataset through the trained model, saving the result to `predictions`

Convert `predictions` to labels using `torch.argmax`, saving the result to `predicted_labels`

Calculate the accuracy of these predicted labels using scikit-learn's `accuracy_score`, saving the result to `accuracy`

Generate a summary of classification metrics for each class using scikit-learn's `classification_report`, saving the summary to `report`

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

Work with `X_test` and `y_test` for evaluation. Set `argmax` parameter `dim` to `1`. Scikit-learn's `accuracy_score` and `classification_report` takes the test labels as the first input.

Jupyter Help
Having trouble testing your work? Double-check that you have followed the steps below to write, run, save, and test your code!
Click here for a walkthrough GIF of the steps below
Run all initial cells to import libraries and datasets. Then follow these steps for each question:

1. Add your solution to the cell with `## YOUR SOLUTION HERE ##`.
2. Run the cell by selecting the `Run` button or the `Shift`+`Enter` keys.
3. Save your work by selecting the `Save` button, the `command`+`s` keys (Mac), or `control`+`s` keys (Windows).
4. Select the `Test Work` button at the bottom left to test your work.

```
import numpy as np
import pandas as pd

import torch
import torch.nn as nn
import torch.optim as optim
```

Import and Prepare Data
Run the code cell to import the data.
We've also created the new multiclass target column `Performance_Outcome` with the `3` target labels:

- Letter grades `0` and `1` are considered **Below Average** (target label `0`)
- Letter grades `2` and `3` are considered **Average** (target label `1`)
- Letter grades `4` and `5` are considered **Above Average** (target label `2`)

Do not change any of our code, as that will impact our testing later!

```
# Load the dataset
df = pd.read_csv("student_performances_encoded.csv")
```

```
# Create Performance Outcome target column {0: Below Average, 1: Average, 2: Above Average}
df['Performance_Outcome'] = df['Letter_Grade'].replace({0:0, 1:0,
                                                        2:1, 3:1,
                                                        4:2, 5:2})

# Preview the dataset
df.head()
```

5 rows × 59 columns

Checkpoint 1/1
We've created a list of columns to use as input features for our model.
Create a tensor `X` from the columns in `train_features`.
Create a tensor `y` from the column `Performance_Outcome` that contains the target labels.
Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
# Creating list of training features
remove_cols = ['Student_ID', 'Letter_Grade', 'Outcome', 'Performance_Outcome']
train_features = [x for x in df.columns if x not in remove_cols]

## YOUR SOLUTION HERE ##

# Create float tensor of input features
X = torch.tensor(df[train_features].values, dtype=torch.float)
# Create long tensor of multiclass targets
y = torch.tensor(df['Performance_Outcome'].values, dtype=torch.long)

# show output - do not modify
X
```

```
tensor([[1., 0., 2.,  ..., 0., 0., 1.],
        [1., 0., 2.,  ..., 0., 0., 1.],
        [1., 0., 2.,  ..., 0., 1., 0.],
        ...,
        [0., 1., 3.,  ..., 0., 1., 0.],
        [1., 1., 3.,  ..., 0., 1., 0.],
        [0., 1., 4.,  ..., 0., 1., 0.]])
```

Create the Training and Testing Sets
Run the cell to split `X` and `y` into training and testing sets using scikit-learn's `train_test_split`.
Do not change any of our code, as that will impact our testing later!

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    train_size=0.8, # use 80% of the data for training
                                                    test_size=0.2, # use 20% of the data for testing
                                                    random_state=42) # set a random state
```

Checkpoint 2/3
Let's train a multiclass neural network!
First, instantiate the loss function and optimizer:

1. instantiate PyTorch's `nn.CrossEntropyLoss()` module and save it to the variable `loss`
2. assign the **stochastic gradient descent optimizer** with a learning rate of `0.01` to the variable `optimizer`

Next, create the training loop:

3. train the network for `1000` epochs on the training set `X_train`
4. calculate the loss using `y_train` and save the loss to the variable `CELoss`

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
from sklearn.metrics import accuracy_score

# set a random seed - do not modify
torch.manual_seed(42)

# define a model
model = nn.Sequential(
    nn.Linear(55, 240),
    nn.ReLU(),
    nn.Linear(240, 110),
    nn.ReLU(),
    nn.Linear(110, 3)
)

## YOUR SOLUTION HERE ##
loss = nn.CrossEntropyLoss()
optimizer = optim.SGD(model.parameters(), lr=0.01)
```

```
# Train the neural network
num_epochs = 1000
for epoch in range(num_epochs):
    predictions = model(X_train)
    CELoss = loss(predictions, y_train)
    CELoss.backward()
    optimizer.step()
    optimizer.zero grad()

    ## DO NOT MODIFY ##
    # keep track of the loss and accuracy during training
    if (epoch + 1) % 100 == 0:
        predicted_labels = torch.argmax(predictions, dim=1)
        accuracy = accuracy_score(y_train, predicted_labels)
        print(f'Epoch [{epoch+1}/{num_epochs}], CELoss: {CELoss.item():.4f}, Accuracy: {accuracy.item():.4f}')
```

```
Epoch [100/1000], CELoss: 0.9913, Accuracy: 0.4828
Epoch [200/1000], CELoss: 0.9345, Accuracy: 0.5431
Epoch [300/1000], CELoss: 0.8730, Accuracy: 0.5431
Epoch [400/1000], CELoss: 0.8031, Accuracy: 0.6552
Epoch [500/1000], CELoss: 0.7237, Accuracy: 0.7414
Epoch [600/1000], CELoss: 0.6393, Accuracy: 0.7845
Epoch [700/1000], CELoss: 0.5586, Accuracy: 0.8103
Epoch [800/1000], CELoss: 0.4874, Accuracy: 0.8276
Epoch [900/1000], CELoss: 0.4263, Accuracy: 0.8793
Epoch [1000/1000], CELoss: 0.3735, Accuracy: 0.8879
```

Checkpoint 3/3

Let's evaluate our trained neural network on the testing set.
We've already set the model to evaluation mode and turned off the gradient calculations.
Add code to:

1. Feed the test dataset through the trained model, saving the result to `predictions`
2. Convert `predictions` to labels using `torch.argmax`, saving the result to `predicted_labels`
3. Calculate the accuracy of these predicted labels using scikit-learn's `accuracy_score`, saving the result to `accuracy`
4. Generate a summary of classification metrics for each class using scikit-learn's `classification_report`, saving the summary to `report`

Don't forget to run the cell and save the notebook before selecting `Test Work`! Open the `Jupyter Help` toggle at the top of the notebook for more details.

```
from sklearn.metrics import accuracy_score, classification_report

model.eval()
with torch.no_grad():
    ## YOUR SOLUTION HERE ##
    predictions = model(X_test)
    predicted_labels = torch.argmax(predictions, dim=1)
    accuracy = accuracy_score(y_test, predicted_labels)
    report = classification_report(y_test, predicted_labels)

# show output - do not modify
print(f'Accuracy: {accuracy.item():.4f}')
print(report)
```

```
Accuracy: 0.6207
              precision    recall  f1-score   support

           0       0.43      0.43      0.43         7
           1       0.67      0.75      0.71        16
           2       0.75      0.50      0.60         6

    accuracy                           0.62        29
   macro avg       0.62      0.56      0.58        29
weighted avg       0.63      0.62      0.62        29
```

Note that while our model had very high accuracy on the training data (89%), its accuracy is much lower on the testing dataset (62%).
The detailed classification report tells us how the model did on each label in its first three rows. For example:

- the model does a poor job of classifying below average students (label `0`) with an F1 score of 43%
- the model does a good job of classifying average students (label `1`) with an F1 score of 71%
-
- the model does an okay job of classifying above average students (label `2`) with an F1 score of 60%
- since our classes are imbalanced (different numbers of students in each class), we'll use the weighted average F1 score of 62% to quantify our model's overall performance

This is why it is so important to evaluate on a testing dataset! This is an example of **overfitting**: our model learned the training dataset too well, in a sense, and so it can't perform as well on data it hasn't seen that might behave a bit differently.
Advanced techniques to address overfitting are outside the scope of this course.
That said, here are some ways you can try improving the model:

- change the training features
- change the number of nodes in the hidden layers
- increase/decrease the number of training epochs

- test different activation functions
- test different optimizers and learning rates

Review

Great work! You've finished this lesson on classification models in PyTorch, building both binary and multiclass models.

Specifically, you've learned how to

prepare and encode categorical data

use sigmoid activation and thresholds for binary classification

use softmax and argmax for multiclass classification

use cross-entropy to compute categorical loss

evaluate network performance using measures like accuracy and F1-scores

This is no small achievement!

Instructions

We've loaded a workspace with the same student performance dataset, and a notebook guiding you through the steps of creating more neural networks for classification! Feel free to explore the dataset and build a model of your own to review everything you've learned.
When you are ready to move on, select `Up Next`.
Happy coding!

https://www.codecademy.com/courses/py-torch-for-classification/lessons/py-torch-classification/exercises/review

Import Libraries

Import `torch`, `pandas`, and any other libraries you might need.

In [ ]:

Import the Dataset

Import the `student_performances_encoded.csv` dataset.

In [ ]:

Explore the Dataset

Explore the different features in the dataset. If needed,refer to `student_performances_raw.csv` to see the unencoded fields.

Are there any classification problems you would like to try out? Pick a target label to move forward with!

Create Tensors

Create a tensor with input features for your model, and another tensor with just the target labels. Remember that the target labels are expected to be integers starting at `0`.

In [ ]:

Train-test-split

Use scikit-learn to create a training dataset and a testing dataset.

In [ ]:

Build and Train the Model

Create a model using `Sequential`. Select appropriate loss and optimizer functions. Create and run a training loop.

In [ ]:

Evaluate the model

Test the trained model on the testing dataset. How did you do? Feel free to go back and iterate on your decisions to try to improve your model performance!

In [ ]:

Share Your Results

Got a cool model built? Share it widely!

In [ ]:

What's Next

What's next?

Congratulations on completing PyTorch for Classification! In this course, you learned about building classification models in PyTorch and are now able to:

Now that you have finished your learning journey, you may be asking, "What's next?" Here are our recommendations for next steps:

Intro to AI Transformers

Take your PyTorch skills to the next level, fine-tuning transformer models with Hugging Face

Intermediate Machine Learning

Level up in machine learning with this intermediate skill path. Learn about hyperparameter tuning, ensemble methods, and more.

Once again, congratulations on finishing your course! We're excited to see what you accomplish next.